

LAB ASSIGNMENT REPORT

MODULE 1 – AUTOMATION WITH SELENIUM

Name: Saheli Deb

Enrollment No.: 12023052019071

Course: Python Automation

Report Type: Section-wise Practical / Lab Report

Software & Environment Requirements

Python 3, Selenium 4, Google Chrome or another supported browser, a code editor/IDE and an active internet connection are required for the examples below. Install Selenium with **python -m pip install selenium**. Modern Selenium versions can use Selenium Manager for browser-driver management.

Important: URLs used in this report are public practice/demo sites. A real lab submission should replace them with the URLs specified by the instructor if a different application is prescribed.

Experiment 1 – Selenium WebDriver Installation and Browser Launch

Aim: To install Selenium for Python, create a WebDriver session, open a web page and verify the browser title.

Requirements: Python 3, Selenium 4, supported browser and internet connection.

Procedure

1. Open a terminal or command prompt.
2. Install Selenium using pip.
3. Create a Chrome WebDriver object.
4. Navigate to a public web page.
5. Print the page title and close the browser.

Source Code

```
# Install first:
# python -m pip install selenium

from selenium import webdriver

driver = webdriver.Chrome()

try:
    driver.get("https://example.com")
    print("Page title:", driver.title)
    assert "Example Domain" in driver.title
finally:
    driver.quit()
```

Expected Result: Expected result: Chrome opens Example Domain, the title is printed, the assertion passes and the browser closes.

Conclusion: Thus, Selenium WebDriver is successfully installed and a browser session is created and terminated programmatically.

Experiment 2 – Locators and Login Automation

Aim: To identify web elements using ID, NAME and XPath locators and automate a login workflow.

Requirements: Python 3, Selenium 4, supported browser and internet connection.

Procedure

1. Open the SauceDemo login page.
2. Locate the username field using ID.
3. Locate the password field using ID/NAME.
4. Locate the login button using ID or XPath.
5. Enter valid credentials and click Login.
6. Verify that the inventory page is displayed.

Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()

try:
    driver.get("https://www.saucedemo.com/")

    driver.find_element(By.ID, "user-name").send_keys("standard_user")
    driver.find_element(By.ID, "password").send_keys("secret_sauce")

    # XPath example
    driver.find_element(
        By.XPATH, "//input[@id='login-button']"
    ).click()

    assert "/inventory.html" in driver.current_url
    print("Login successful")
finally:
    driver.quit()
```

Expected Result: Expected result: The valid user is logged in and the URL changes to the inventory page.

Conclusion: The experiment demonstrates practical use of element locators and WebDriver interactions.

Experiment 3 – Explicit Wait and Dynamic Elements

Aim: To synchronize a test with a dynamically loaded element using WebDriverWait and Expected Conditions.

Requirements: Python 3, Selenium 4, supported browser and internet connection.

Procedure

1. Open the dynamic-loading demo page.
2. Click Start.
3. Wait until the hidden/dynamic element becomes visible.
4. Read the displayed text.
5. Assert the expected text.

Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()

try:
    driver.get(
        "https://the-internet.herokuapp.com/dynamic_loading/1"
    )

    driver.find_element(
        By.CSS_SELECTOR, "#start button"
    ).click()

    wait = WebDriverWait(driver, 10)

    element = wait.until(
        EC.visibility_of_element_located(
            (By.ID, "finish")
        )
    )

    assert element.text == "Hello world!"
    print("Dynamic element verified")
finally:
    driver.quit()
```

Expected Result: Expected result: The script waits until the element is visible instead of using a fixed delay.

Conclusion: Explicit synchronization makes the automation more reliable for asynchronous web pages.

Experiment 4 – Checkboxes and Web Controls

Aim: To automate checkbox controls and verify their selected state.

Requirements: Python 3, Selenium 4, supported browser and internet connection.

Procedure

1. Open the checkbox practice page.
2. Locate all checkbox elements.
3. Select the first checkbox if it is not selected.
4. Deselect the second checkbox if it is selected.
5. Verify the final state.

Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()

try:
    driver.get("https://the-internet.herokuapp.com/checkboxes")

    boxes = driver.find_elements(
        By.CSS_SELECTOR,
        "input[type='checkbox']"
    )

    if not boxes[0].is_selected():
        boxes[0].click()

    if boxes[1].is_selected():
        boxes[1].click()

    assert boxes[0].is_selected()
    assert not boxes[1].is_selected()

    print("Checkbox states verified")
finally:
    driver.quit()
```

Expected Result: Expected result: Checkbox 1 is selected and checkbox 2 is not selected.

Conclusion: The experiment demonstrates state-aware interaction rather than blindly clicking a control.

Experiment 5 – JavaScript Alerts, Confirm and Prompt

Aim: To handle JavaScript alert dialogs by switching from the page context to the alert context.

Requirements: Python 3, Selenium 4, supported browser and internet connection.

Procedure

1. Open the JavaScript alerts demo page.
2. Trigger an alert and accept it.
3. Trigger a confirm dialog and dismiss it.
4. Trigger a prompt, enter text and accept it.
5. Print the resulting page message where applicable.

Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()

try:
    driver.get(
        "https://the-internet.herokuapp.com/javascript_alerts"
    )

    # Alert
    driver.find_element(
        By.XPATH, "//button[text()='Click for JS Alert']"
    ).click()
    driver.switch_to.alert.accept()

    # Confirm
    driver.find_element(
        By.XPATH, "//button[text()='Click for JS Confirm']"
    ).click()
    driver.switch_to.alert.dismiss()

    # Prompt
    driver.find_element(
        By.XPATH, "//button[text()='Click for JS Prompt']"
    ).click()

    prompt = driver.switch_to.alert
    prompt.send_keys("Saheli")
    prompt.accept()

    print("All alert types handled successfully")
finally:
    driver.quit()
```

Expected Result: Expected result: Alert is accepted, confirm is dismissed and prompt receives the supplied text.

Conclusion: The experiment demonstrates switching browser context to JavaScript dialog objects.

Experiment 6 – Web Table Data Extraction

Aim: To locate a table, iterate through rows and extract a particular cell from the row matching a condition.

Requirements: Python 3, Selenium 4, supported browser and internet connection.

Procedure

1. Open the web-table demo page.
2. Locate all body rows.
3. Read each row's cells.
4. Search for a target surname.
5. Extract the required column value.
6. Verify that a matching record was found.

Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()

try:
    driver.get("https://the-internet.herokuapp.com/tables")

    rows = driver.find_elements(
        By.CSS_SELECTOR, "#table1 tbody tr"
    )

    target = "Doe"
    value = None

    for row in rows:
        cells = row.find_elements(By.TAG_NAME, "td")

        if cells and target in cells[0].text:
            value = cells[4].text
            break

    assert value is not None
    print("Extracted value:", value)
finally:
    driver.quit()
```

Expected Result: Expected result: The row containing the target name is found and the required column is extracted.

Conclusion: The experiment demonstrates dynamic table traversal and text-based record identification.

Experiment 7 – Iframe and Multiple Window Handling

Aim: To switch between different browser contexts such as an iframe and a newly opened tab/window.

Requirements: Python 3, Selenium 4, supported browser and internet connection.

Procedure

1. Open the iframe demo page.
2. Locate the iframe and switch into it.
3. Read content from the iframe.
4. Return to the main document.
5. Open a new browser tab using JavaScript.
6. Switch to the new window, read its title and return to the original window.

Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()

try:
    driver.get("https://the-internet.herokuapp.com/iframe")

    frame = driver.find_element(By.ID, "mce_0_ifr")
    driver.switch_to.frame(frame)

    editor = driver.find_element(By.ID, "tinymce")
    print("Iframe content:", editor.text)

    driver.switch_to.default_content()

    original = driver.current_window_handle

    driver.execute_script(
        "window.open('https://example.com', '_blank');"
    )

    new_window = [
        h for h in driver.window_handles
        if h != original
    ][0]

    driver.switch_to.window(new_window)
    print("New window title:", driver.title)

    driver.close()
    driver.switch_to.window(original)

finally:
    driver.quit()
```

Expected Result: Expected result: The script successfully enters the iframe, returns to the main document, switches to the new window and then restores the original window.

Conclusion: The experiment demonstrates browser-context management.

Experiment 8 – Screenshot and Exception Handling

Aim: To capture diagnostic evidence when an automation step fails and guarantee browser cleanup.

Requirements: Python 3, Selenium 4, supported browser and internet connection.

Procedure

1. Start the browser.
2. Open the target page.
3. Perform a validation that can fail.
4. Capture a screenshot inside the exception handler.
5. Re-raise the failure so the test runner records it.
6. Always close the browser using finally.

Source Code

```
from selenium import webdriver

driver = webdriver.Chrome()

try:
    driver.get("https://www.saucedemo.com/")

    # Deliberately normal validation
    assert "Swag Labs" in driver.title

except Exception as exc:
    driver.save_screenshot("failure.png")
    print("Test failed:", exc)
    raise

finally:
    driver.quit()
```

Expected Result: Expected result: On failure, failure.png is generated and the browser is closed even if an exception occurs.

Conclusion: The experiment demonstrates basic failure diagnostics and resource cleanup.

Experiment 9 – Data-Driven Selenium Test Using CSV

Aim: To separate test data from automation logic and execute multiple test cases from a CSV file.

Requirements: Python 3, Selenium 4, supported browser and internet connection.

Procedure

1. Create a CSV file with username, password and expected result columns.
2. Read each row using csv.DictReader.
3. Pass each row's values into the Selenium workflow.
4. Validate the expected outcome.
5. Keep test data outside the Python automation code.

Source Code

```
# Example CSV:
# username,password,valid
# standard_user,secret_sauce,True
# wrong_user,wrong_pass,False

import csv

with open("login_data.csv", newline="",
          encoding="utf-8") as file:

    reader = csv.DictReader(file)

    for row in reader:
        username = row["username"]
        password = row["password"]
        expected = row["valid"]

        print(
            "Running:",
            username,
            "Expected:",
            expected
        )

    # The values can now be supplied to
    # the Selenium login workflow.
```

Expected Result: Expected result: Multiple test records are read without duplicating the Selenium test logic.

Conclusion: The experiment demonstrates the foundation of data-driven automation.

Experiment 10 – Mouse Hover and JavaScript Interaction

Aim: To perform advanced pointer interaction and execute a small JavaScript operation when appropriate.

Requirements: Python 3, Selenium 4, supported browser and internet connection.

Procedure

1. Open the target application.
2. Locate a hover menu or interactive element.
3. Move the pointer to the element using ActionChains.
4. Scroll the element into view using JavaScript.
5. Continue with normal Selenium interactions.

Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.action_chains import ActionChains

driver = webdriver.Chrome()

try:
    driver.get("https://example.com")

    element = driver.find_element(
        By.TAG_NAME, "h1"
    )

    ActionChains(driver).move_to_element(
        element
    ).perform()

    driver.execute_script(
        "arguments[0].scrollIntoView({block: 'center'});",
        element
    )

    print("Advanced interaction completed")
finally:
    driver.quit()
```

Expected Result: Expected result: The pointer interaction executes and the selected element is brought into view.

Conclusion: The experiment demonstrates ActionChains and controlled JavaScript execution.

Overall Module 1 Lab Conclusion

The practical work in Module 1 demonstrates the complete basic workflow of Selenium automation: environment setup, browser control, locator strategies, element interaction, explicit synchronization, advanced browser controls, table extraction, context switching, screenshots, exception handling and data-driven execution. These experiments provide the practical foundation required to build maintainable Selenium test suites in later modules.

Lab Submission Checklist

Item	Status / Evidence
Selenium installed	Python Selenium package installed
Browser automation	WebDriver launches and closes browser
Locators	ID / XPath / CSS demonstrated
Synchronization	Explicit WebDriverWait demonstrated
Controls	Checkboxes and JavaScript dialogs handled
Advanced contexts	Iframe and window switching demonstrated
Web table	Rows and cells extracted
Error handling	Screenshot + try/except/finally demonstrated
Test data	CSV-driven execution demonstrated